

Développement front-end avancé



Références d'enquêtes annuelles :

<https://survey.stackoverflow.co/>

<https://stateofjs.com/en-US>

Partie 1: Rappel HTML, CSS et JavaScript



Le Web est construit avec HTML, CSS et JavaScript.

HTML



Structure et contenu d'une page Web

CSS



Mettre en forme une page web (Design)

JavaScript



- **Structure dynamique** d'une page web (**DOM**)
- **Interactivité** d'une page web (**événements**)
- **Requêtes asynchrones** pour récupérer des données (**XMLHttpRequest-XHR**)

JavaScript – Structure dynamique (DOM)

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>HeH - DST</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Bachelier en Informatique</h1>
```

```
  <div>
```

```
    <p>L'équipe pédagogique</p>
```

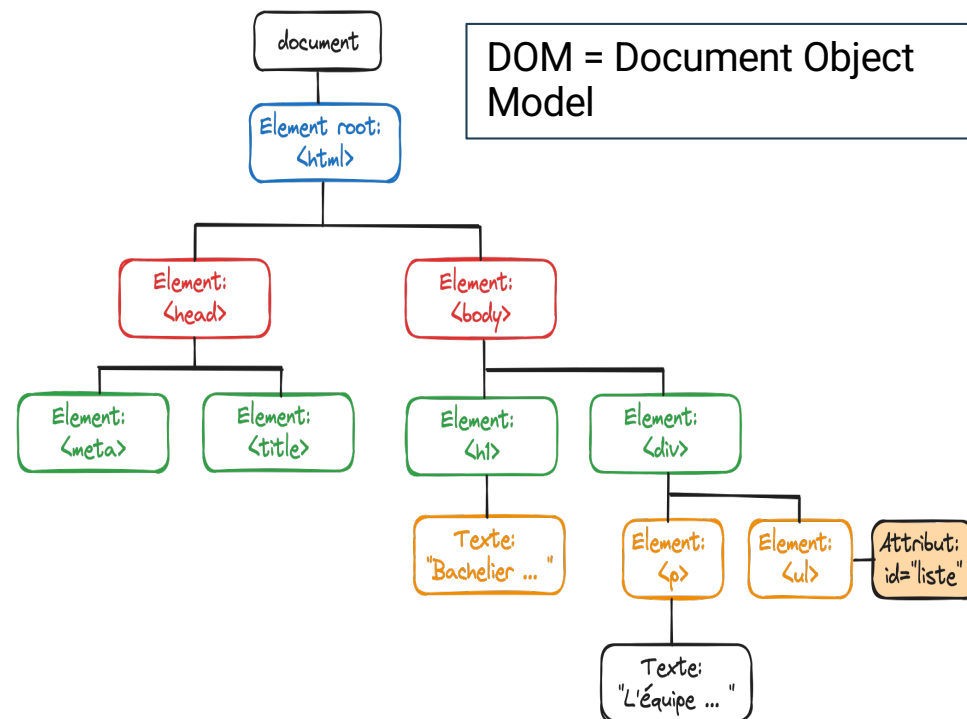
```
    <ul id="liste">
```

```
  </ul>
```

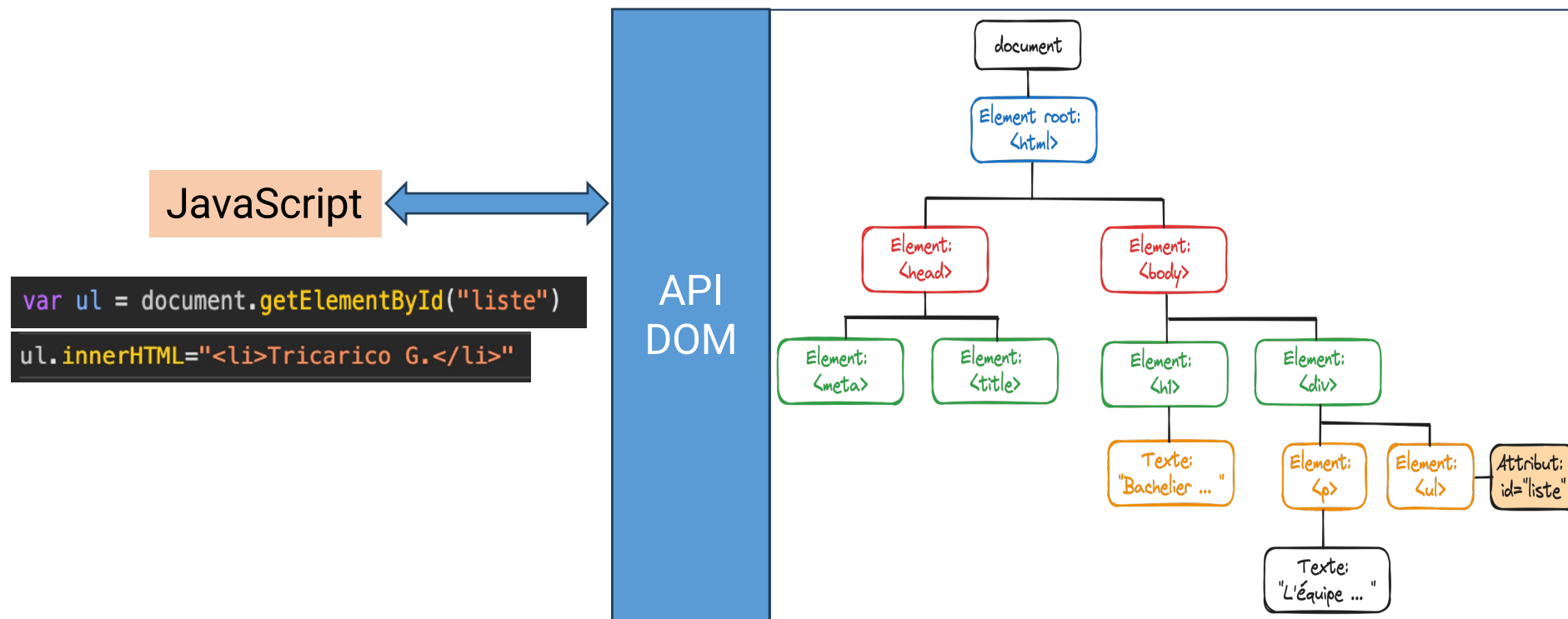
```
  </div>
```

```
</body>
```

```
</html>
```



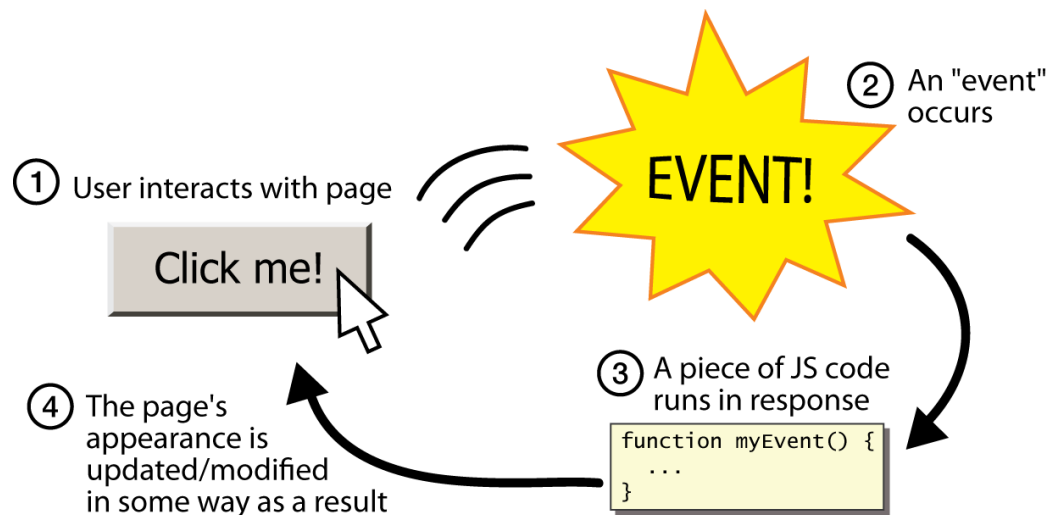
JavaScript – Structure dynamique (DOM)



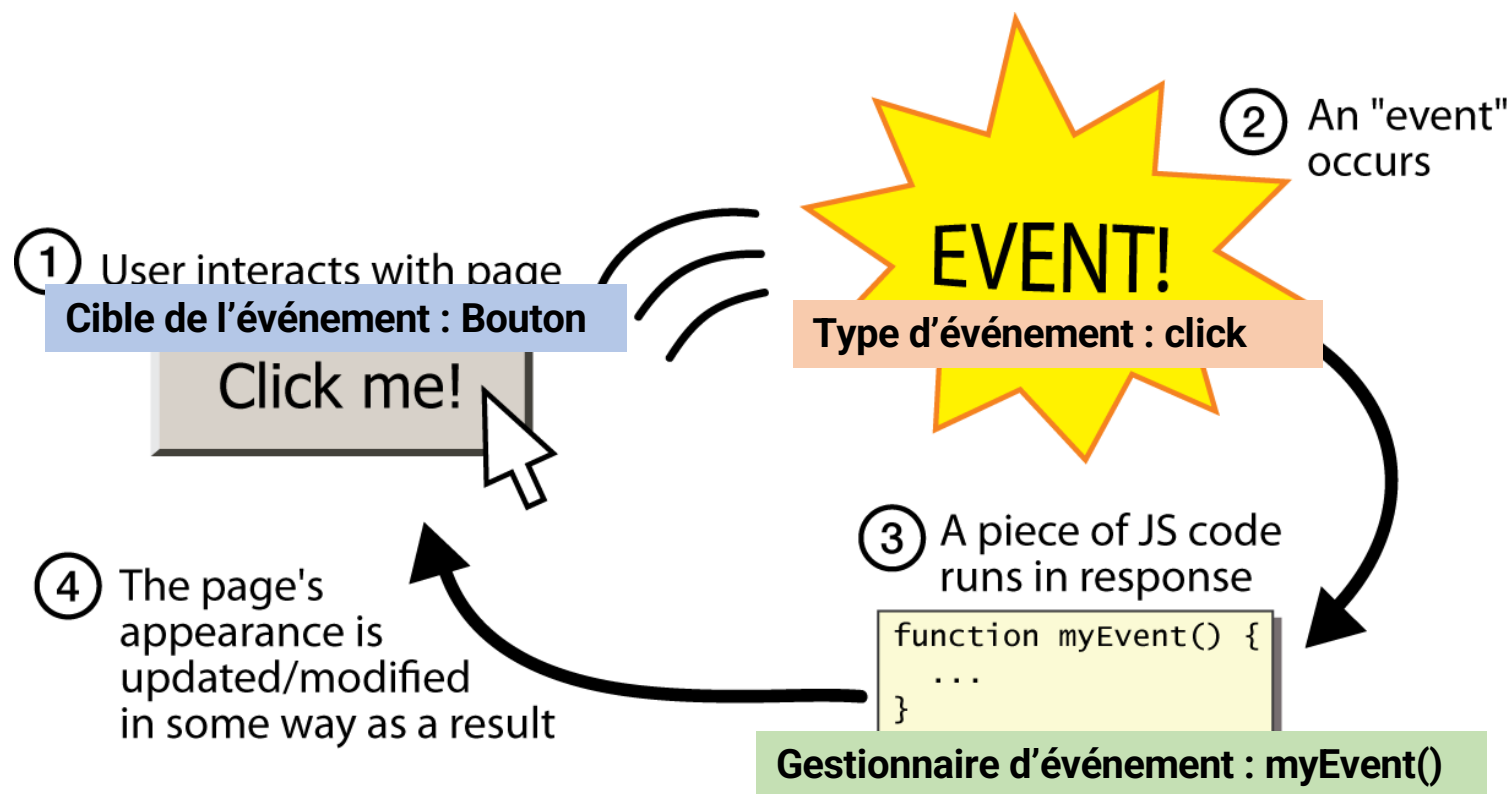
La méthode **getElementById()** de Document renvoie un objet Element représentant l'élément dont la propriété id correspond à la chaîne de caractères spécifiée.

La propriété **Element.innerHTML** de Element récupère ou définit la syntaxe HTML décrivant les descendants de l'élément.

- Toutes les applications dotées d'une interface utilisateur graphique peuvent enregistrer une ou plusieurs fonctions à invoquer lorsque des événements spécifiques se produisent
- Les événements peuvent se produire sur n'importe quel **élément** d'un document HTML.



- **Type d'événement (*event type*)**
C'est une **chaîne de caractères** qui indique le type d'événement (click, load, ...)
- **Cible de l'événement (*event target*)**
C'est **l'objet** sur lequel l'événement s'est produit ou avec lequel l'événement est associé (bouton, champ texte, ...)
- **Gestionnaire d'événement ou écouteur d'événement (event handler, or *event listener*)**
Cette **fonction** gère ou répond à un événement.
- **Objet événement (*event object*)**
Cet objet est associé à un événement particulier et contient des détails sur cet événement. Les objets d'événement sont transmis en tant qu'argument à la fonction de gestion d'événement.



Il existe trois méthodes de base pour enregistrer des gestionnaires d'événements.

1. Spécifier les événements avec HTML

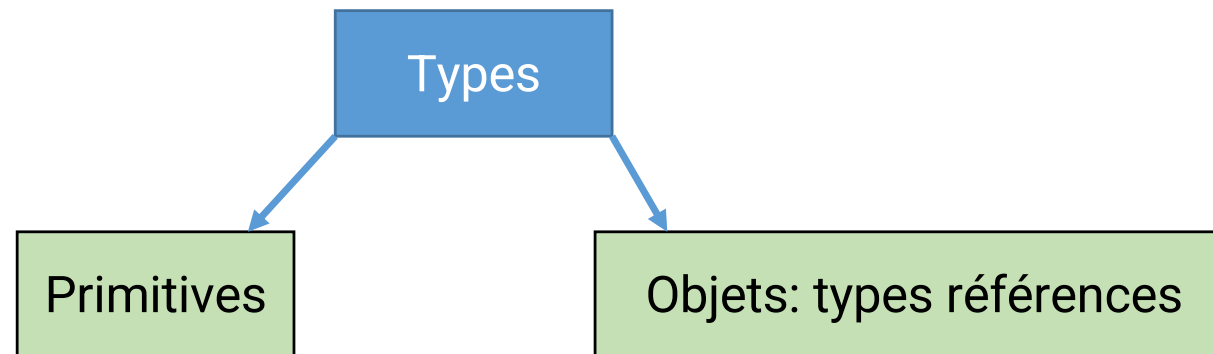
```
<button id=« mybutton » onclick=«handleClick() »>Click me</button>
```

2. Spécifier les événements avec JavaScript

```
<script>  
  let b = document.getElementById("mybutton");  
  b.onclick = function() { console.log("Thanks for clicking me!"); };  
</script>
```

3. Spécifier les événements avec « Event Listener » => Basé sur le pattern observer

```
b.addEventListener("click", function() { console.log("Thanks again!"); });
```



- boolean
- number
- string
- null
- undefined

- functions
- arrays
- class
- object

Elles se composent du mot-clé ***function***, suivi des éléments suivants :

- Un **identifiant** qui nomme la fonction. Obligatoire => **le nom de la fonction devient une variable dont la valeur est la fonction elle-même.**
- Une **paire de parenthèses** autour d'une liste de zéro ou plusieurs **paramètres**. => ils se comportent comme des variables locales dans le corps de la fonction.
- Une **paire d'accolades** contenant les instructions JavaScript. Ces instructions constituent le **corps de la fonction** : elles sont exécutées chaque fois que la fonction est invoquée.

Les déclarations de fonctions sont évaluées avant que le reste du code ne soit interprété.

Remarquez la différence entre *paramètres* et *arguments* :

- Les paramètres d'une fonction sont les noms listés dans la définition de la fonction.
- Les arguments d'une fonction sont les valeurs réelles passées à la fonction.
- Les paramètres sont initialisés avec les valeurs des arguments fournis.

The diagram shows a code snippet on a dark background. The function definition is `function sum(param1, param2){` followed by `return param1 + param2;` and a closing brace. An orange bracket labeled "Parameters" points to `param1` and `param2`. Below the function, the call `sum(5, 6);` is shown. A blue bracket labeled "Arguments" points to the values `5` and `6`.

```
function sum(param1, param2){  
  return param1 + param2;  
}  
  
sum(5, 6);
```

<https://developer.mozilla.org/fr/docs/Glossary/Parameter>

```
let pt1 = {  
  x : 2,  
  y : 3  
}  
let pt2 = {  
  x : 4,  
  y : 5  
}  
  
function distanceEntre2Points(point1,point2){  
  let deltaX = point2.x - point1.x;  
  let deltaY = point2.y - point1.y;  
  return Math.sqrt(deltaX*deltaX + deltaY*deltaY);  
}  
  
//Invoquer la fonction => mettre les parenthèses  
let resultat = distanceEntre2Points(pt1,pt2);  
console.log('distanceEntre2Points(pt1,pt2) -> ' + resultat);  
//Affecter la référence de la fonction "distanceEntre2Points" à la variable "distance"  
let distance = distanceEntre2Points;  
console.log('distance(pt1,pt2) -> ' + distance(pt1,pt2));
```

Les **expressions de fonction** ressemblent beaucoup aux déclarations de fonction, mais elles apparaissent dans le contexte d'une expression et leur nom est facultatif.

```
const carre = function carreNombre (nombre){  
  return nombre*nombre;  
}  
  
console.log('Fonction nommée -> ' + carre(2));  
  
// le nom de la fonction n'est pas utile  
// => on peut le supprimer => fonction anonyme  
  
const carreAnonyme = function (nombre){  
  return nombre*nombre;  
}  
  
console.log('Fonction anonyme -> ' + carreAnonyme(2));
```

Bonne pratique : Utiliser **const** avec les **expressions de fonction** afin de ne pas écraser accidentellement vos fonctions en leur assignant de nouvelles valeurs.

La plupart des fonctions définies comme des **expressions n'ont pas besoin de noms**, ce qui rend leur définition plus compacte.

```
//Le nom de la fonction est utile pour les appels récursifs
const facto = function fact(x) {
  if (x <= 1) return 1;
  else return x*fact(x-1); };

console.log('La factorielle de 5 -> ' + facto(5));
```


La **déclaration d'une fonction** peut être faite après l'appel :

```
console.log(carre(5));  
/* ... */  
function carre(n) { return n*n }
```

Déclaration de fonction

≠

≠

```
console.log(carre2);  
// ReferenceError: carré2 is not defined  
console.log(carre2(5));  
// TypeError: carre2 is not a function  
  
let carre2 = function (n) {  
  return n * n;  
}
```

Expression de fonction

En **ES6**, vous pouvez définir des fonctions à l'aide d'une **syntaxe** particulièrement **compacte** appelée "**fonctions fléchées** ».

```
const somme = function (nbre1,nbre2){  
  return nbre1+nbre2;  
}  
  
//Fonction fléchée avec deux paramètres  
const sommeFl = (nbre1,nbre2) => {return nbre1+nbre2;}  
  
// Fonction fléchée simplifiée  
const sommeFl2 = (nbre1,nbre2) => nbre1+nbre2;  
  
console.log(somme(5,2));  
console.log(sommeFl(5,2));  
console.log(sommeFl2(5,2));
```

```
const carre = function(nbre) {  
  return nbre*nbre;  
}  
  
//fonction fléchée avec un paramètre  
const carreFl = nbre => nbre*nbre;  
  
console.log(carreFl(2));
```

```
const afficheBonjour = function() {  
  return console.log('Bonjour')  
}  
//fonction fléchée sans paramètre  
const afficheBonjourFl = () => console.log('Bonjour');  
  
afficheBonjour();  
afficheBonjourFl();
```

La syntaxe concise des fonctions fléchées est lorsque vous avez besoin de passer une fonction à une autre, ce qui est courant avec des méthodes de tableau comme *map()*, *filter()* et *reduce()*.

Partie 2: Introduction à React



Introduction - React



Lorsque vous utilisez une **bibliothèque**, vous importez son code et appelez ses fonctions.

Cette bibliothèque **JavaScript** (frontend) va permettre :

- De construire des interfaces utilisateurs **réactives (React)**
- Des composants réutilisables

1. **Interface réactive** (d'où le nom React)
=> **DOM virtuel** (cf *reactSansJSX_DOMVirtuel.zip*)
2. **Composants**
=> Éléments de décomposition d'une page web => blocs de construction.
3. **Déclarative** (la philosophie de React étant que vous déclarez votre ou vos états finaux souhaités et que vous laissez React déterminer comment y parvenir)
=> **JSX**

React - DOM virtuel

Voir le code : DOMVirtuel_JavaScript_avecEtSansReact.zip

index.html

```
<body>
  <div id="Conteneur1"></div>
  <div id="Conteneur2"></div>
  <script
    src="https://unpkg.com/react@16/umd/react.development.js"
    crossorigin
  ></script>
  <script
    src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"
    crossorigin
  ></script>

  <!-- Chargement du JavaScript -->
  <script src="script.js"></script>
```

Importe la bibliothèque React

Importe la bibliothèque ReactDOM

React - DOM virtuel

```
const e = React.createElement;
const domContainer1 = document.getElementById("Conteneur1");
const domContainer2 = document.getElementById("#Conteneur2");

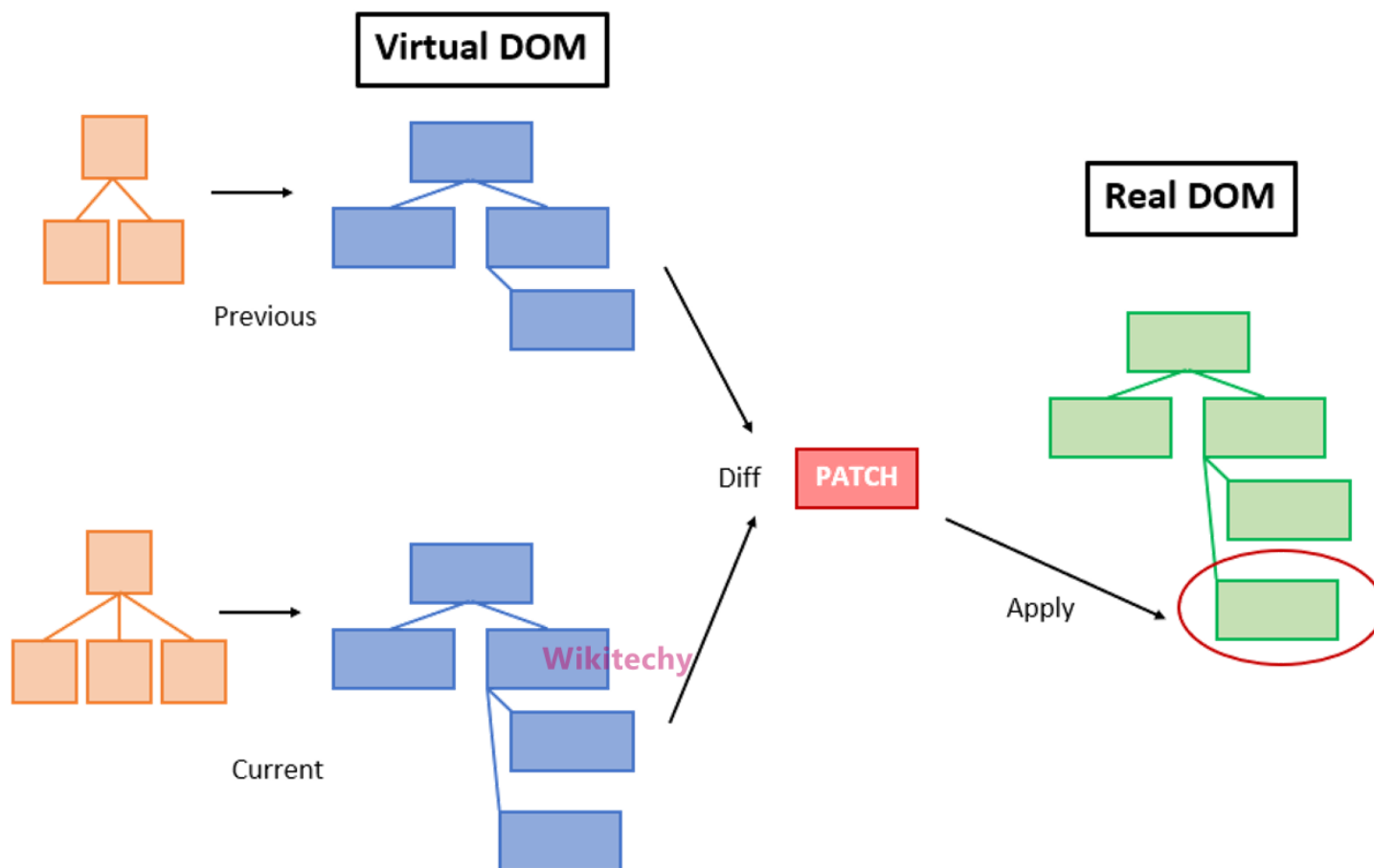
const render = () => {
  //Utilisation de Javascript sans React
  domContainer1.innerHTML = `
    <div>
      Hello HTML
      <input />
      <pre>${new Date().toLocaleString()}</pre>
    </div>
  `;

  -----

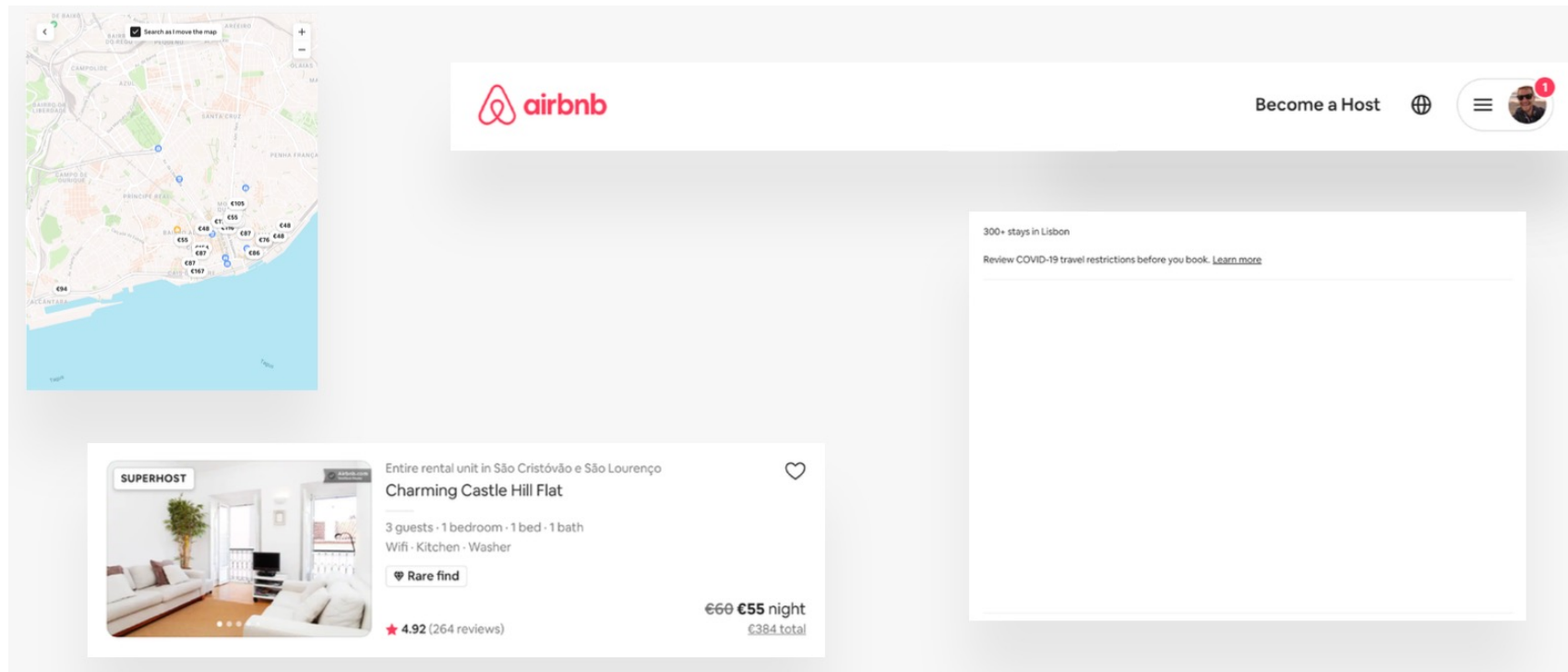
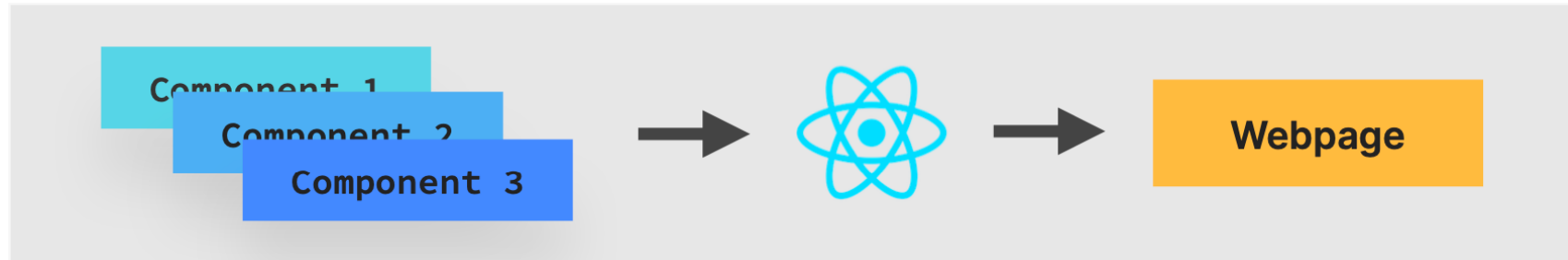
  //Utilisation de JavaScript avec React
  const elementsReact = React.createElement(
    "div",
    null,
    "Hello React ",
    React.createElement("input", null),
    React.createElement("pre", null, new Date().toLocaleString(
  );
  ReactDOM.render(elementsReact, Conteneur2);
};

setInterval(render, 1000);
```


React - DOM virtuel



React – Composants



React – Composants


NavBar

Lisbon
Oct 9 – 16, 2023
Add guests

Search

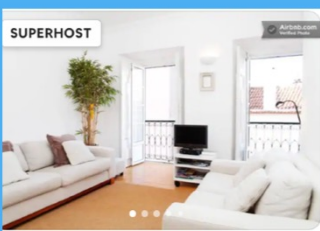
Become a Host




Price
Type of place
Free cancellation
Beachfront
Wifi
Kitchen
Washer
Pool
Self check-in
Dedicated workspace
Free parking
Iron
Filters

300+ stays in Lisbon
Review COVID-19 travel restrictions before you book. [Learn more](#)

Results



SUPERHOST

Entire rental unit in São Cristóvão e São Lourenço

Charming Castle Hill Flat

3 guests · 1 bedroom · 1 bed · 1 bath

Wifi · Kitchen · Washer

 Rare find

★ 4.92 (264 reviews)

€60 ~~€55~~ night

€384 total

Listing



SUPERHOST

Entire rental unit in São Vicente de Fora

 **Feel Alfama at BCA | Cascao 18**

2 guests · 1 bedroom · 1 bed · 1 bath

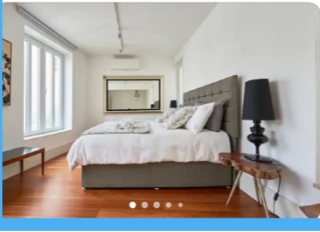
Wifi · Kitchen · Washer · Air conditioning

★ 4.92 (252 reviews)

€48 night

€333 total

Listing



Entire rental unit in Alfama

Duplex & Terrace with a river view

4 guests · 1 bedroom · 2 beds · 1.5 baths

Wifi · Kitchen · Washer · Air conditioning

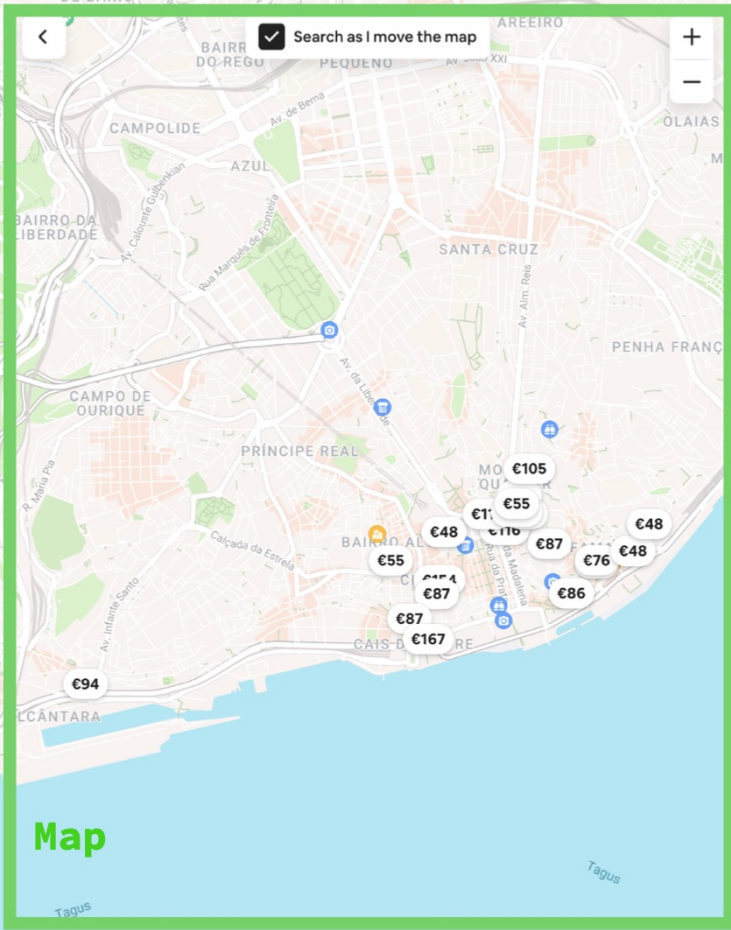
★ 4.98 (44 reviews)

€93 ~~€86~~ night

€597 total

Listing

☒ Search as I move the map



Map

React – JSX (déclarative)

JSX (JavaScript XML) : JSX est une extension de syntaxe pour JavaScript qui vous permet d'écrire du balisage **similaire** au **HTML** au sein d'un fichier **JavaScript**.

Nous décrivons l'aspect et le fonctionnement des composants à l'aide d'une syntaxe déclarative appelée JSX

```
1 const elementsReact = (  
2   <div>  
3     Hello React  
4     <input />  
5     <pre>{new Date().toLocaleString()}</pre>  
6   </div>  
7 );
```

JSX => syntaxe déclarative

JavaScript => syntaxe impérative

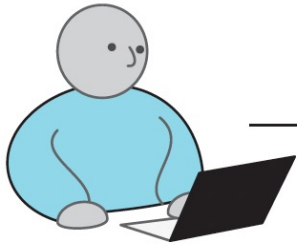
```
1 const elementsReact = /*#__PURE__*/ React.createElement(  
2   "div",  
3   null,  
4   "Hello React",  
5   /*#__PURE__*/ React.createElement("input", null),  
6   /*#__PURE__*/ React.createElement("pre", null, new Date().toLocaleString())  
7 );
```

React – JSX (déclarative)

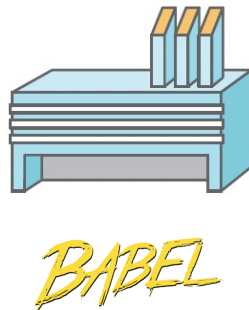
```
1 const elementsReact = (  
2   <div>  
3     Hello React  
4     <input />  
5     <pre>{new Date().toLocaleString()}</pre>  
6   </div>  
7 );
```

```
1 const elementsReact = /*#__PURE__*/ React.createElement(  
2   "div",  
3   null,  
4   "Hello React",  
5   /*#__PURE__*/ React.createElement("input", null),  
6   /*#__PURE__*/ React.createElement("pre", null, new Date().toLocaleString())  
7 );
```

1. JSX



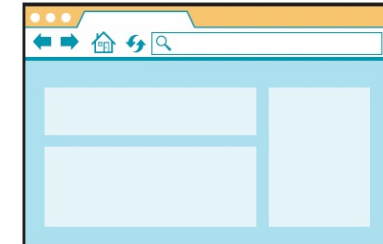
2. Transpiler



3. JavaScript

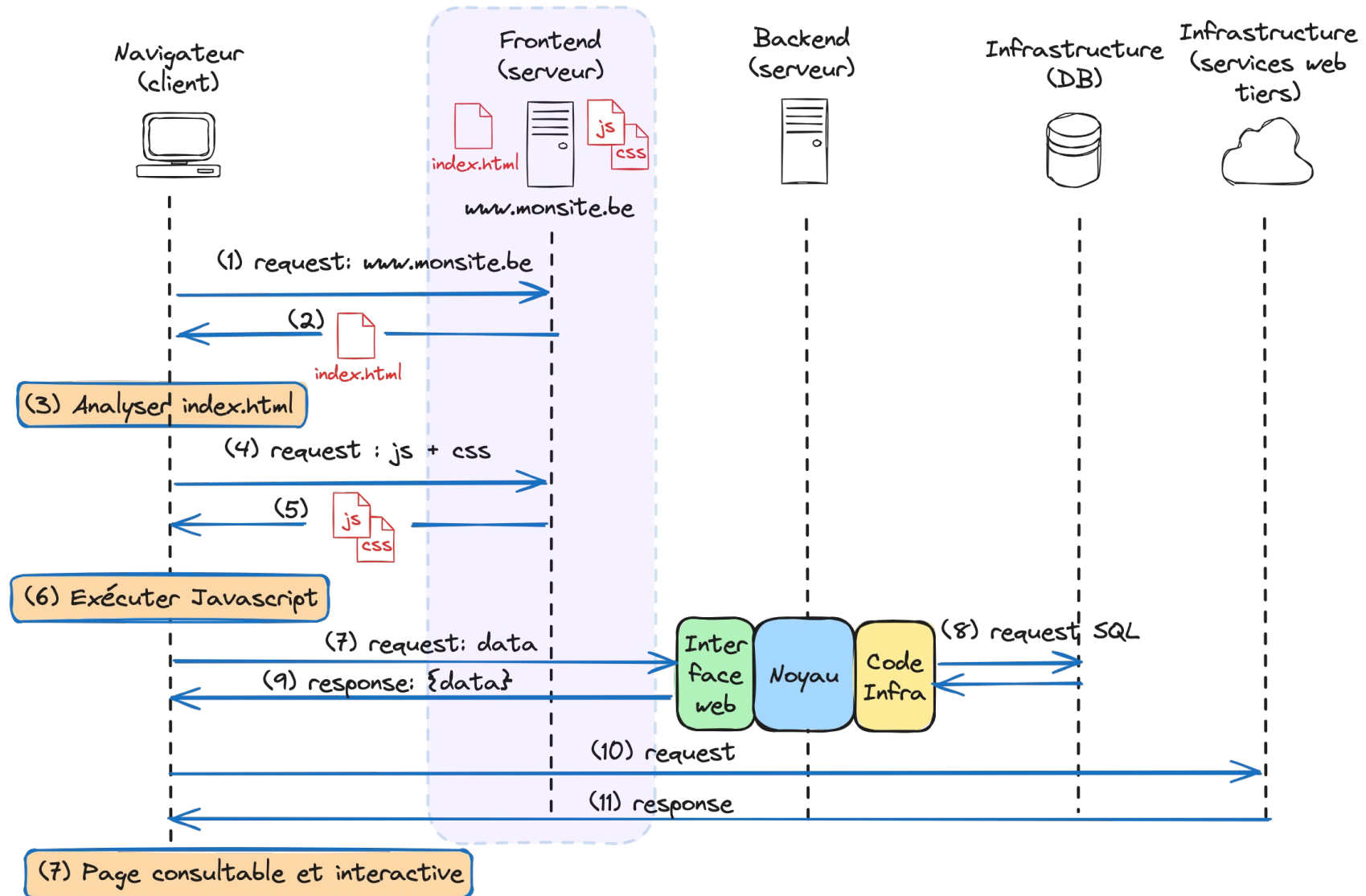


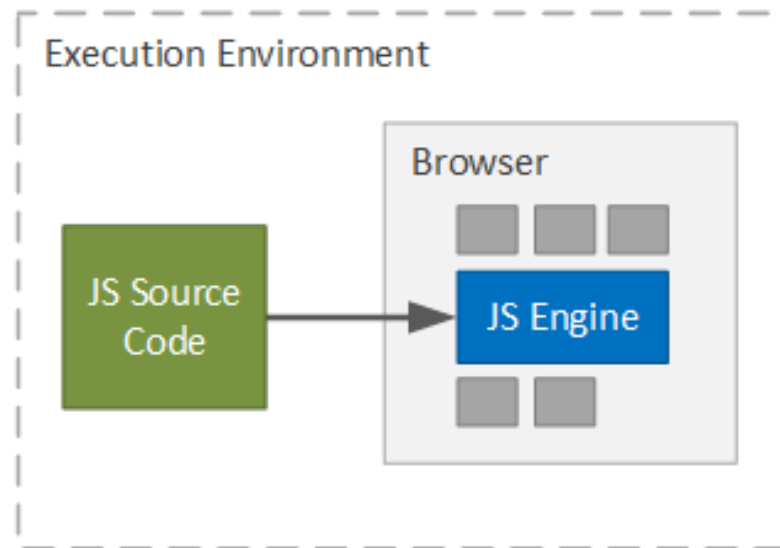
4. Browser



<https://babeljs.io/>

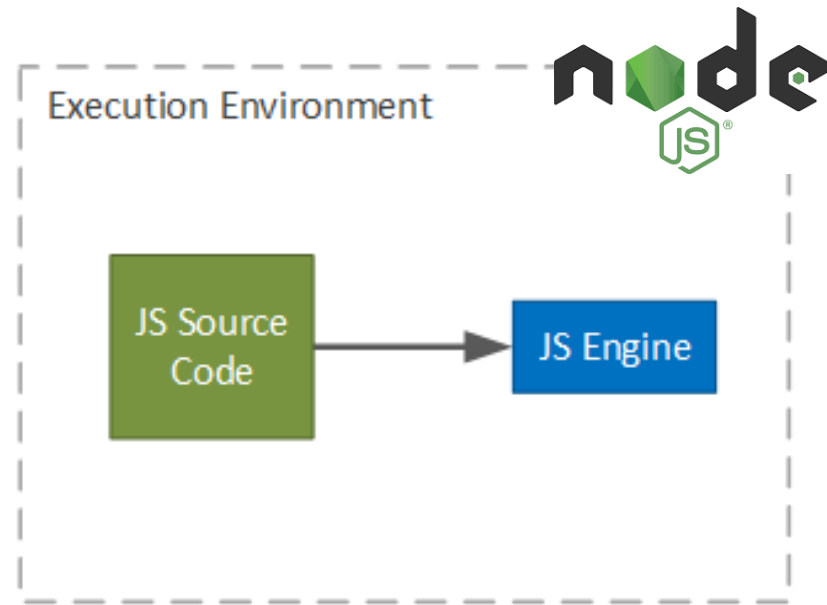
React – Single Page applications (SPA)





JAVASCRIPT IN A BROWSER

```
<script src=« ./app.js »></script>
```



STANDALONE JAVASCRIPT

```
$ node app.js
```



V8 est un **moteur JavaScript (JS) open-source** développé par Google.



« **Vite** est un **serveur de développement local** écrit par Evan You, le créateur de Vue.js, et utilisé par défaut par Vue et pour les modèles de projets React. Il prend en charge TypeScript et JSX. » (Wikipédia)

Vite nécessite l'installation de *Node.js* et *npm* (gestionnaire de paquets)

<https://nodejs.org/en>

```
node --version  
v21.6.1
```

```
npm --version  
10.2.4
```

Mettre à jour Node.js : <https://www.npmjs.com/package/n>

Introduction – Créer un projet React

```
npm create vite@latest ex1 -- --template react
Need to install the following packages:
create-vite@5.2.0
Ok to proceed? (y) y

Scaffolding project in /Users/giannitricarico/Library/Developer/
Done. Now run:

  cd ex1
  npm install
  npm run dev
```

npm run dev ➡ Lancer le serveur de développement

```
> ex1@0.0.0 dev
> vite

VITE v5.1.0 ready in 696 ms

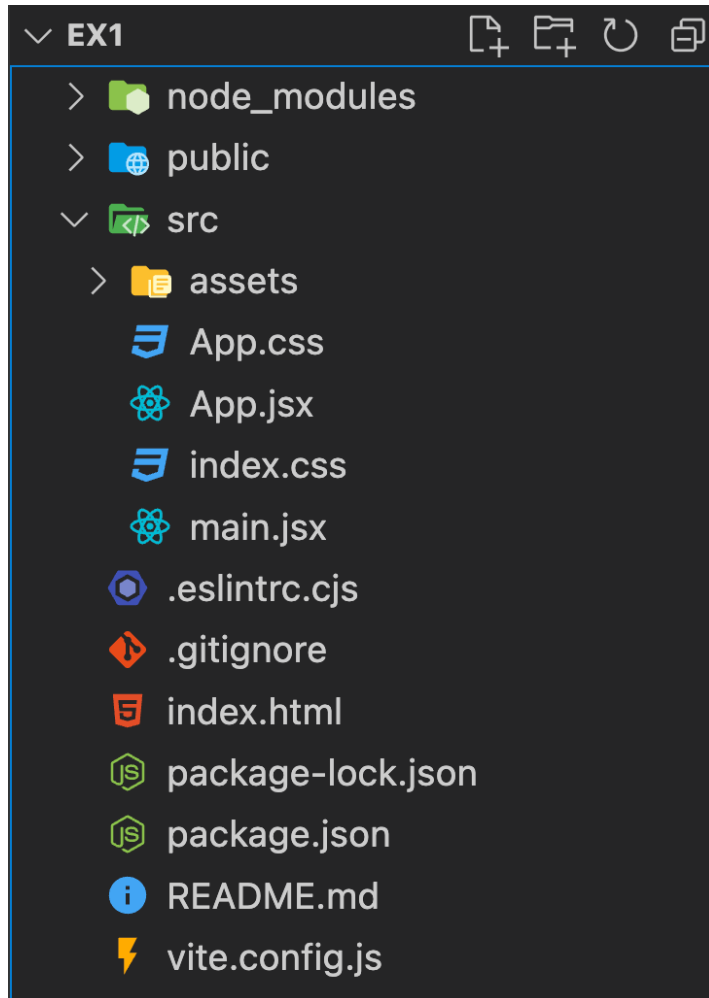
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

The supported template presets are:

JavaScript	TypeScript
vanilla	vanilla-ts
vue	vue-ts
react	react-ts
preact	preact-ts
lit	lit-ts
svelte	svelte-ts
solid	solid-ts
qwik	qwik-ts

<https://vitejs.dev/guide/>

Introduction – Créer un projet React



package.json : Ce fichier contient une liste de toutes les dépendances tierces.

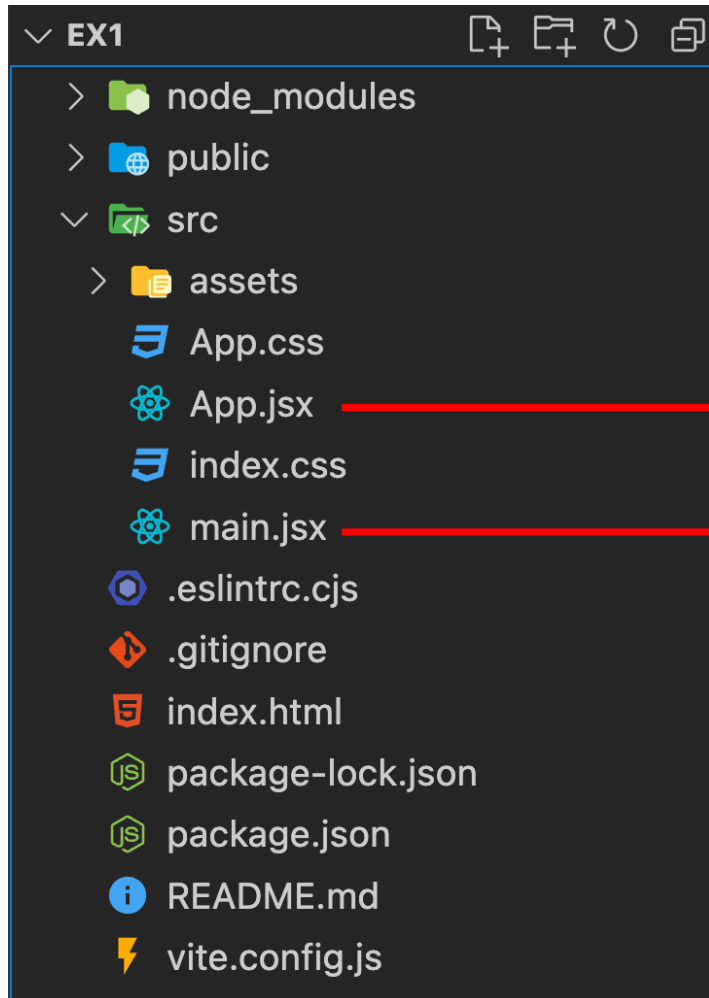
node_modules/ : Ce dossier contient tous les paquets node qui ont été installés. Comme nous avons utilisé Vite pour créer notre application React, plusieurs modules node (par exemple React) sont déjà installés pour nous.

vite.config.js : Un fichier pour configurer Vite.

public/ : Ce dossier contient des ressources statiques pour le projet, comme une favicon qui est utilisée pour la vignette de l'onglet du navigateur lors du démarrage du serveur de développement ou de la construction du projet pour la production.

index.html : Le HTML qui est affiché dans le navigateur au démarrage du projet.

Introduction – Créer un projet React



src/App.jsx qui est utilisé pour implémenter les composants React => **composant root**.

src/main.jsx est le point d'entrée d'une application React.

Toutes les commandes spécifiques à votre projet se trouvent dans votre fichier package.json

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint . --ext js,jsx",  
  "preview": "vite preview"  
}
```

Ces scripts sont exécutés avec la commande : `npm run <script>`

#Exemples

Runs the application locally for the browser (start dev server)

`npm run dev`

Builds the application (production)

`npm run build`

locally preview production build

`npm run preview`

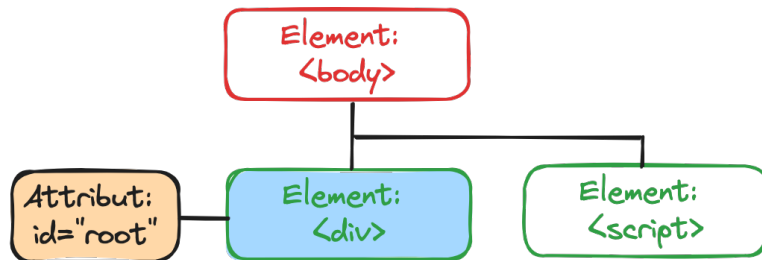
Introduction – Principe de React

index.html

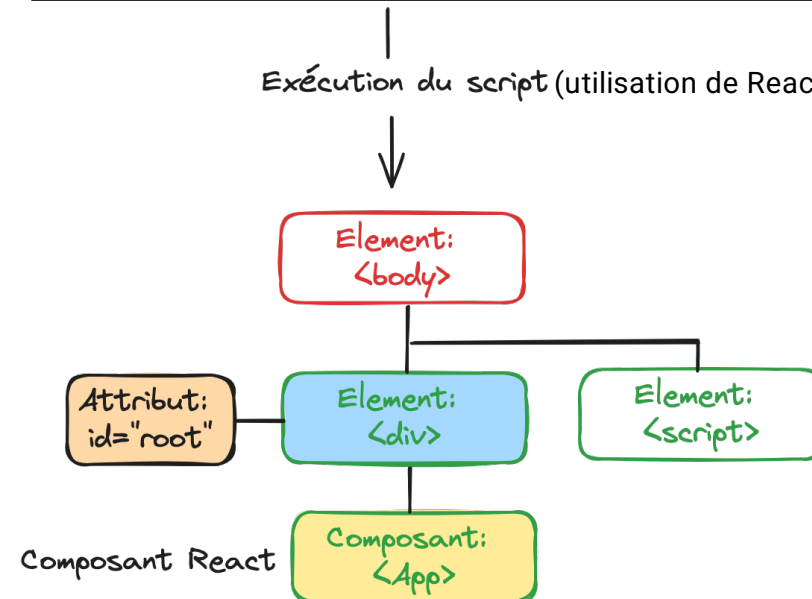
```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

main.jsx

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

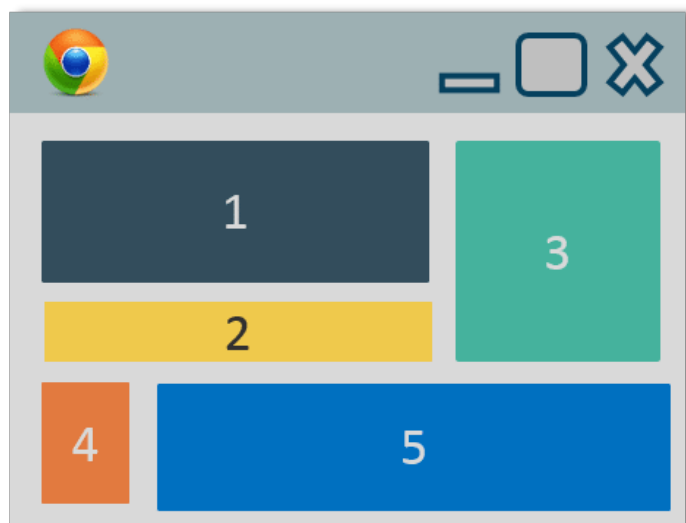


Exécution du script (utilisation de ReactDOM)

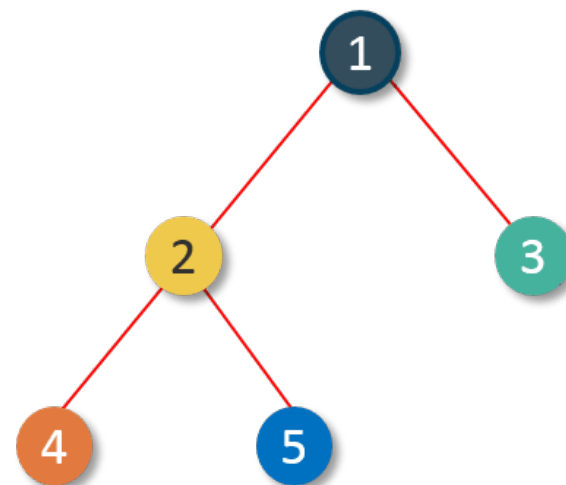


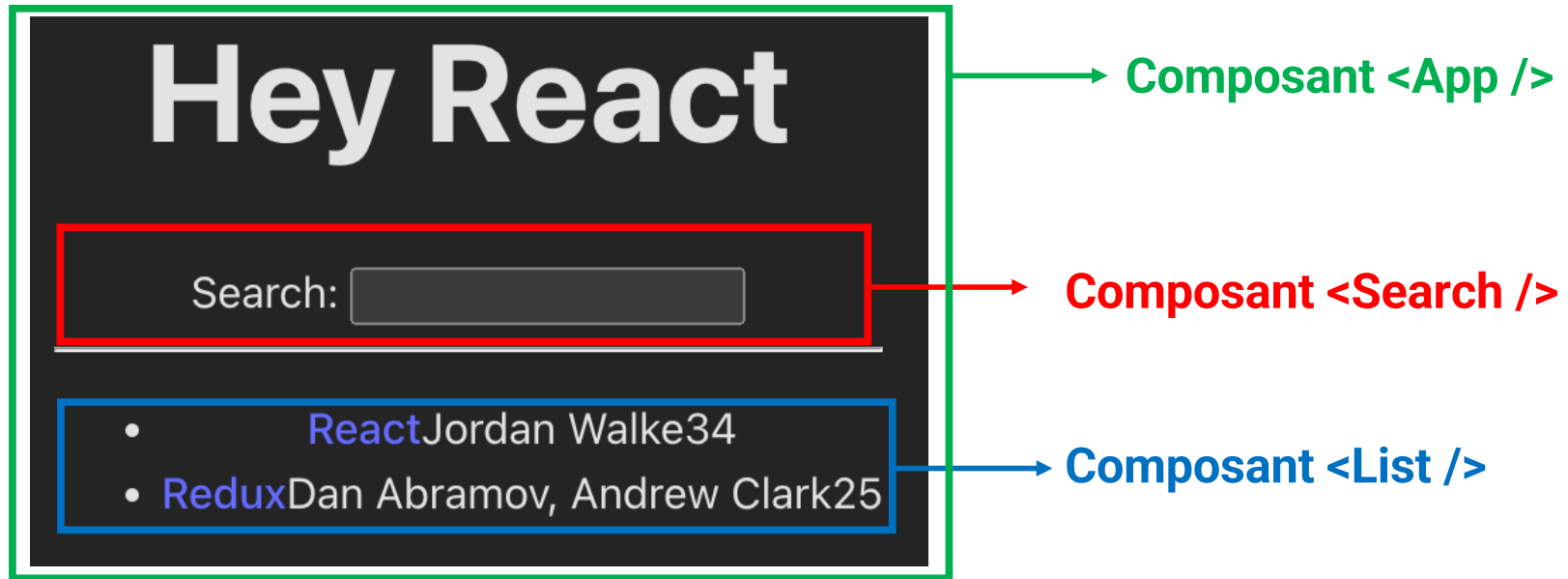
Partie 3: Composants et props

Browser



UI Tree





2 types de composant :

- Composants **intégrés** => <p>, <div>, ...
- Composant **personnalisé** en React => utilisé comme élément HTML dans du code JSX

Les composants personnalisés sont des fonctions JavaScript qui :

- **renvoient** une valeur pouvant être **rendue par React** (typiquement du code JSX)
- portent des noms en **majuscules** (sauf les **composants intégrés** de React comme <p>, <div>,...)
- sont **exportées** (via *export default*)


```
1  import "./App.css";
2
3  function App() {
4      return (
5          <div>
6              <h1>Hello React</h1>
7          </div>
8      );
9  }
10
11 export default App;
```

Composant de fonction « App »

Retourne JSX => Rendu par le navigateur

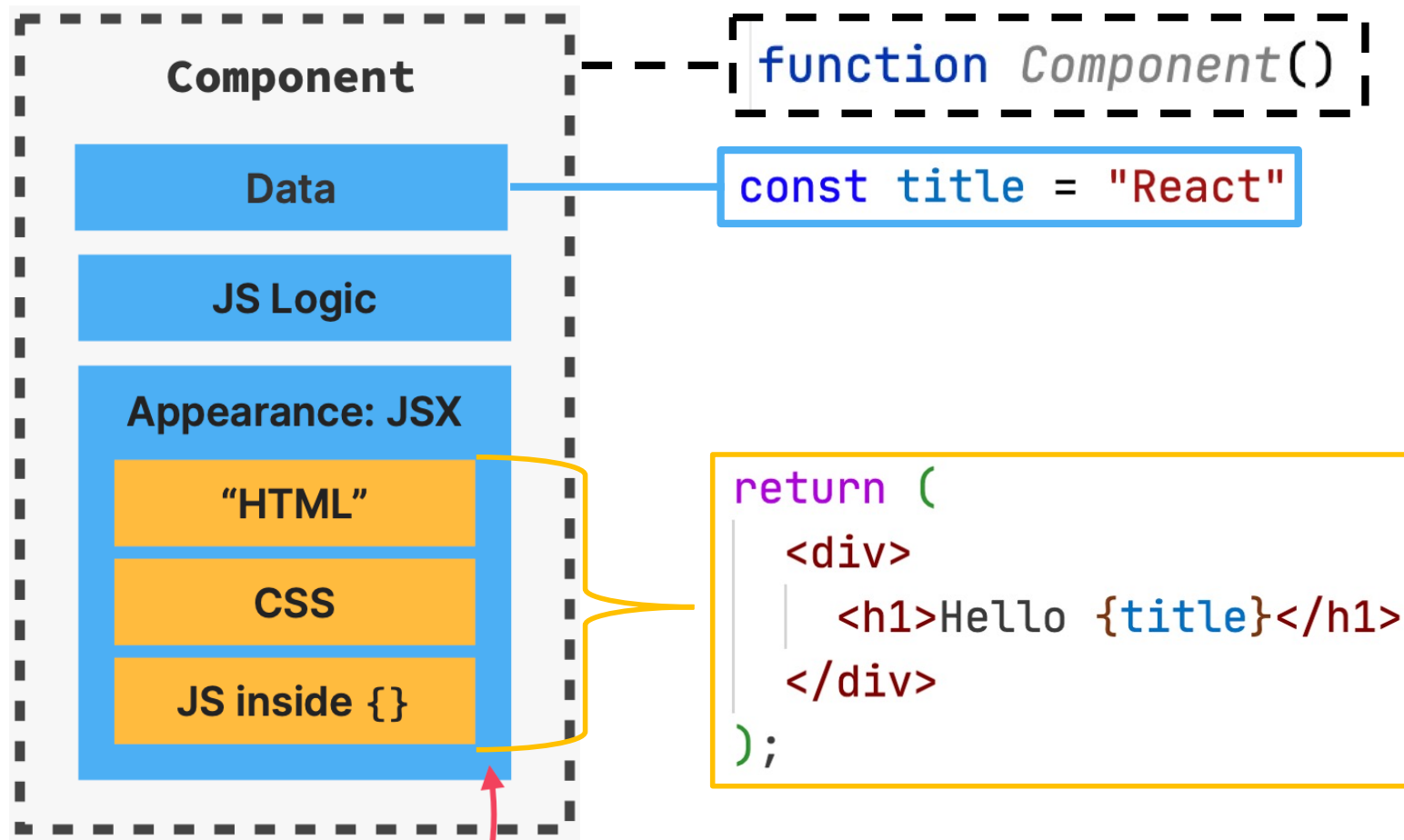
Exporter la fonction (module)

Un **composant** doit commencer par une **majuscule**, sinon il n'est pas traité comme un composant dans React.

Contenu dynamique

```
3  function App() {  
4  |    const title = "React"  
5  |    return (  
6  |        <div>  
7  | |    <h1>Hello {title}</h1>  
8  |    </div>  
9  |    );  
10 | }
```

1 composant = 1 fonction

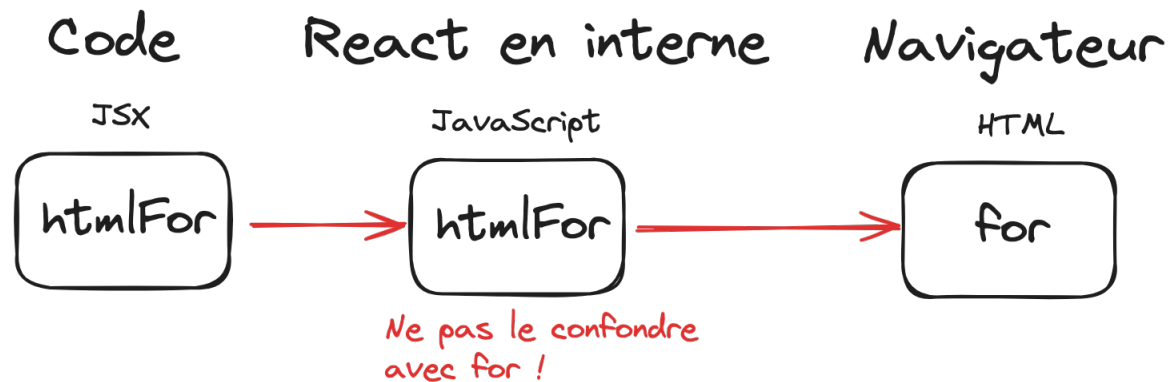


```

3  function App() {
4    const title = "React"
5    return (
6      <div>
7        <h1>Hello {title}</h1>
8        <label htmlFor="search">Search: </label>
9        <input id="search" type="text" />
10     </div>
11   );
12 }

```

JSX !!!!



React utilise la convention de dénomination **camelCase** pour les attributs.
Exemple : **className** au lieu de **class** et **onClick** au lieu de **onclick**

Composant `<App />`

```
import './App.css'

function App() { Show usages
  const welcome : {greeting: string, title: string} = {
    greeting: 'Hey',
    title: 'React'
  };
  return (
    <h1> {welcome.greeting} {welcome.title}</h1>
  )
}

export default App Show usages
```

Hey React

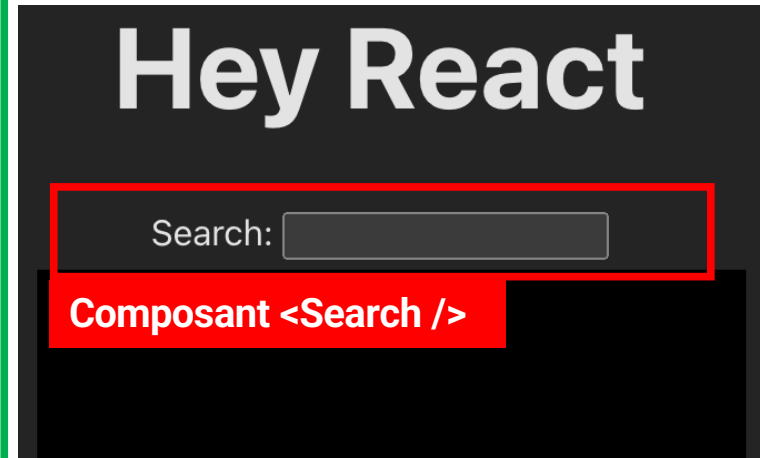
```
function App() { Show usages
  const welcome : {greeting: string, title: string} = {
    greeting: 'Hey',
    title: 'React'
  };
  return (
    <div>
      <h1> {welcome.greeting} {welcome.title}</h1>
      <Search/> ← Utilisation du composant « Search »
    </div>
  )
}

const Search = () => { ← Définition du composant « Search »
  return (
    <div>
      <label htmlFor="search">Search: </label>
      <input id="search" type="text"/>
    </div>
  );
};

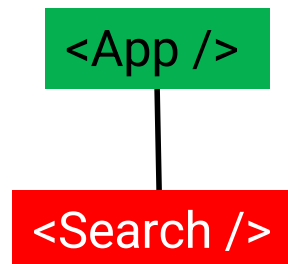
export default App Show usages
```

1 composant = 1 fonction

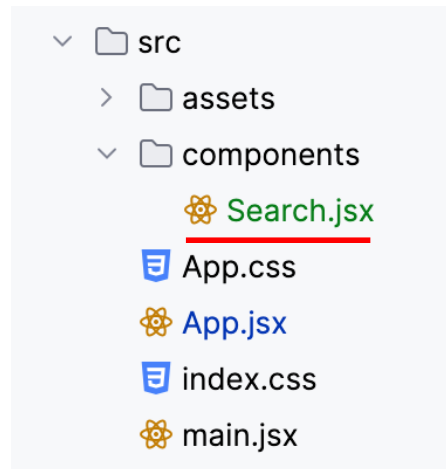
Composant <App />



Arbre de rendu



Fichier : App.jsx



```
1  const Search = () => { Show usages new *  
2      return (  
3          <div>  
4              <label htmlFor="search">Search: </label>  
5              <input id="search" type="text"/>  
6          </div>  
7      );  
8  };  
9  export default Search; Show usages new *
```

1 composant = 1 fonction = 1 fichier

- src
 - assets
 - components
 - Search.jsx
 - App.css
 - App.jsx
 - index.css
 - main.jsx

```
1 import './App.css'
2 import Search from './components/Search'
3 function App() { Show usages ⓘ HeH-Gianni
4     const welcome : {greeting: string, title: string} = {
5         greeting: 'Hey',
6         title: 'React'
7     };
8     return (
9         <div>
10             <h1> {welcome.greeting} {welcome.title}</h1>
11             <Search/>
12         </div>
13     )
14 }
15 export default App Show usages ⓘ HeH-Gianni
```


Données JavaScript

Object JavaScript

```
1  const book = {  
2      title: 'React',  
3      url: 'https://reactjs.org/',  
4      author: 'Jordan Walke',  
5      num_comments: 3,  
6      points: 4,  
7      objectID: 0  
8  }
```

Tableau

```
19  const numbers = [1, 2, 3, 4];  
20  const shopping = ["bread", "milk", "cheese", "hummus", "noodles"];
```

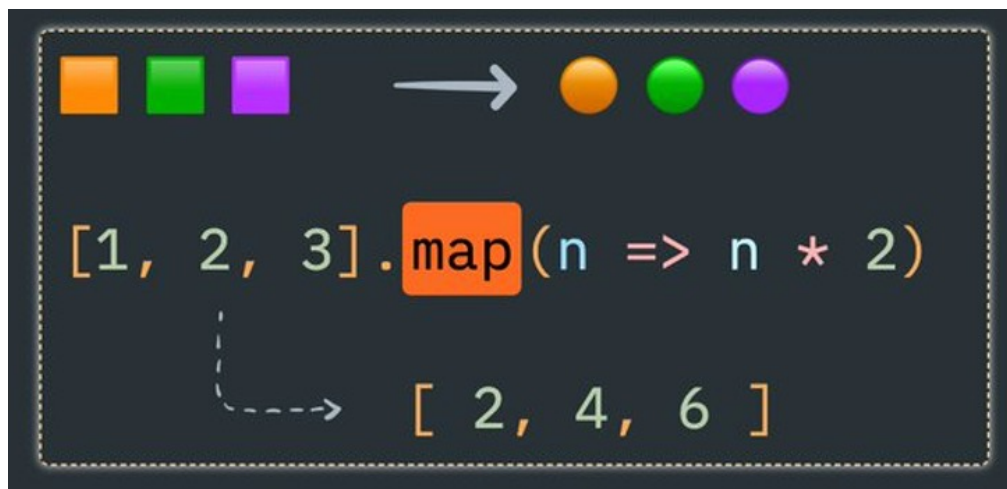
Tableau d'objets JavaScript

```
1  ∨ const books = [  
2  ∨  {  
3      title: "React",  
4      url: "https://reactjs.org/",  
5      author: "Jordan Walke",  
6      num_comments: 3,  
7      points: 4,  
8      objectID: 0,  
9  },  
10 ∨  {  
11     title: "Redux",  
12     url: "https://redux.js.org/",  
13     author: "Dan Abramov, Andrew Clark",  
14     num_comments: 2,  
15     points: 5,  
16     objectID: 1,  
17  },  
18 ];
```

La méthode ***map()*** (« carte » en français) retourne un **nouveau tableau** composé des éléments du tableau initial auxquels une fonction est appliquée.

```
const fruits = ["pomme", "banane", "poire", "kiwi"];  
//const fruitsMaj = [];  
  
// for(let fruit of fruits){  
// fruitsMaj.push(fruit.toUpperCase());  
// }  
  
const fruitsMaj = fruits.map((fruit) => fruit.toUpperCase());  
  
console.log(fruitsMaj);  
// ["POMME", "BANANE", "POIRE", "KIWI"]
```

Synthèse de la méthode (fonction) *map()* :



tab.map(callback)

=> Une fonction de rappel (*callback*) est une fonction passée dans une autre fonction en tant qu'argument, qui est ensuite invoquée à l'intérieur de la fonction externe pour accomplir une sorte de routine ou d'action.

Liste d'éléments ()

```
<hr />
<ul>{books.map((item) => {
  return <li key={item.objectID}> {item.title} </li>
})}</ul>
</div>
```

Une « clé » (*key*), est un **attribut spécial** que vous devez inclure quand vous créez une liste d'éléments.

Les **key** (clés) aident React à **identifier quels éléments d'une liste ont changé, ont été ajoutés ou supprimés**. Vous devez donner une clé à chaque élément dans un tableau, afin d'apporter aux éléments une identité stable.

Vous pouvez aussi **utiliser l'id associée à votre donnée** dans votre base de données.

```
const List = () => { Show usages new *  
  return (  
    <ul>  
      {books.map((book : {...} ) => (  
        <li key={book.objectID}>  
          <span><a href={book.url}>{ book.title}</a></span>  
          <span> {book.author}</span>  
          <span> ({book.num_comments})</span>  
          <span> [{book.points}]</span>  
        </li>  
      ))}  
    </ul>  
  );  
}  
export default List; Show usages new *
```

Hey React

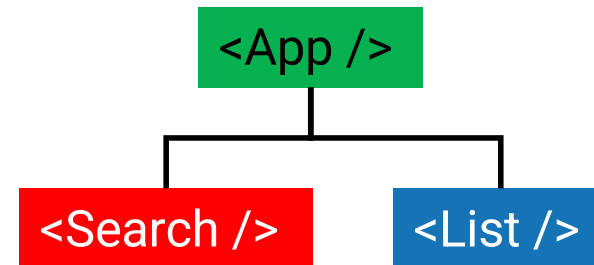
Search:

- [React](#) Jordan Walke (3) [4]
- [Redux](#) Dan Abramov, Andrew Clark (2) [5]

Composant <List />

```
import Search from './components/Search'
import List from './components/List.jsx';
function App() { Show usages ⓘ HeH-Gianni *
  const welcome : {greeting: string, title: string} = {
    greeting: 'Hey',
    title: 'React'
  };
  return (
    <>
      <h1> {welcome.greeting} {welcome.title}</h1>
      <Search/>
      <hr />
      <List />
    </>
  )
}
export default App Show usages ⓘ HeH-Gianni
```

Arbre de rendu



```
function App() {  
  return (  
    <p>Hello World!</p>  
    <p>Let's learn React!</p>  
  );  
};
```

Invalide => Une fonction ne peut que renvoyer une seule valeur !!

```
function App() {  
  return (  
    <React.Fragment>  
      <p>Hello World!</p>  
      <p>Let's learn React!</p>  
    </React.Fragment>  
  );  
}
```

Valide

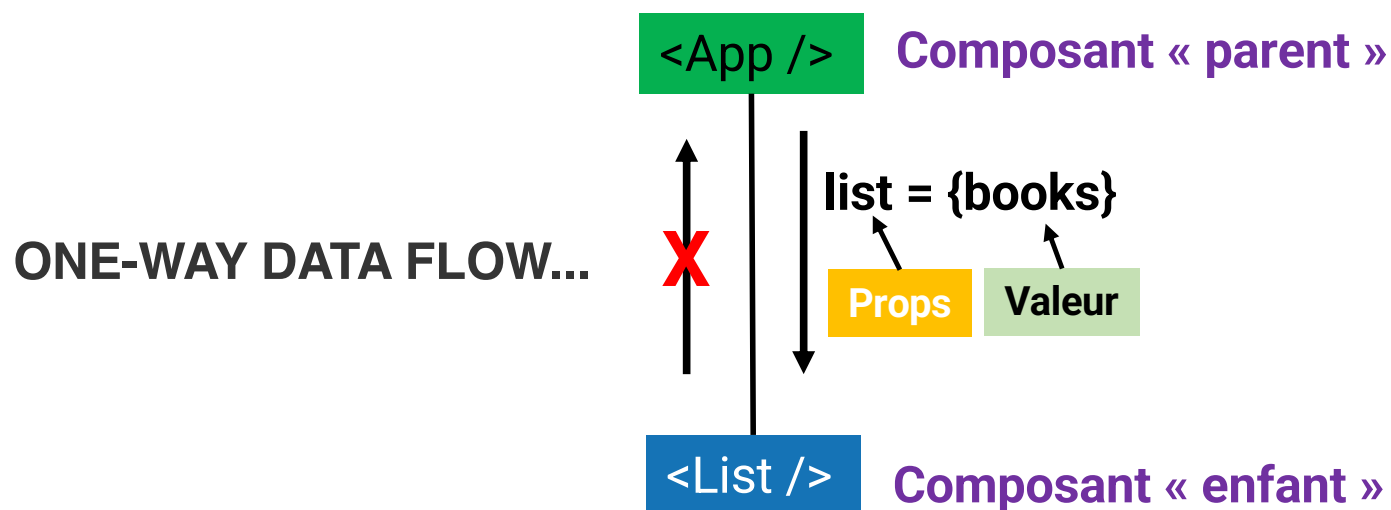
```
function App() {  
  return (  
    <div>  
      <p>Hello World!</p>  
      <p>Let's learn React!</p>  
    </div>  
  );  
}
```

Valide

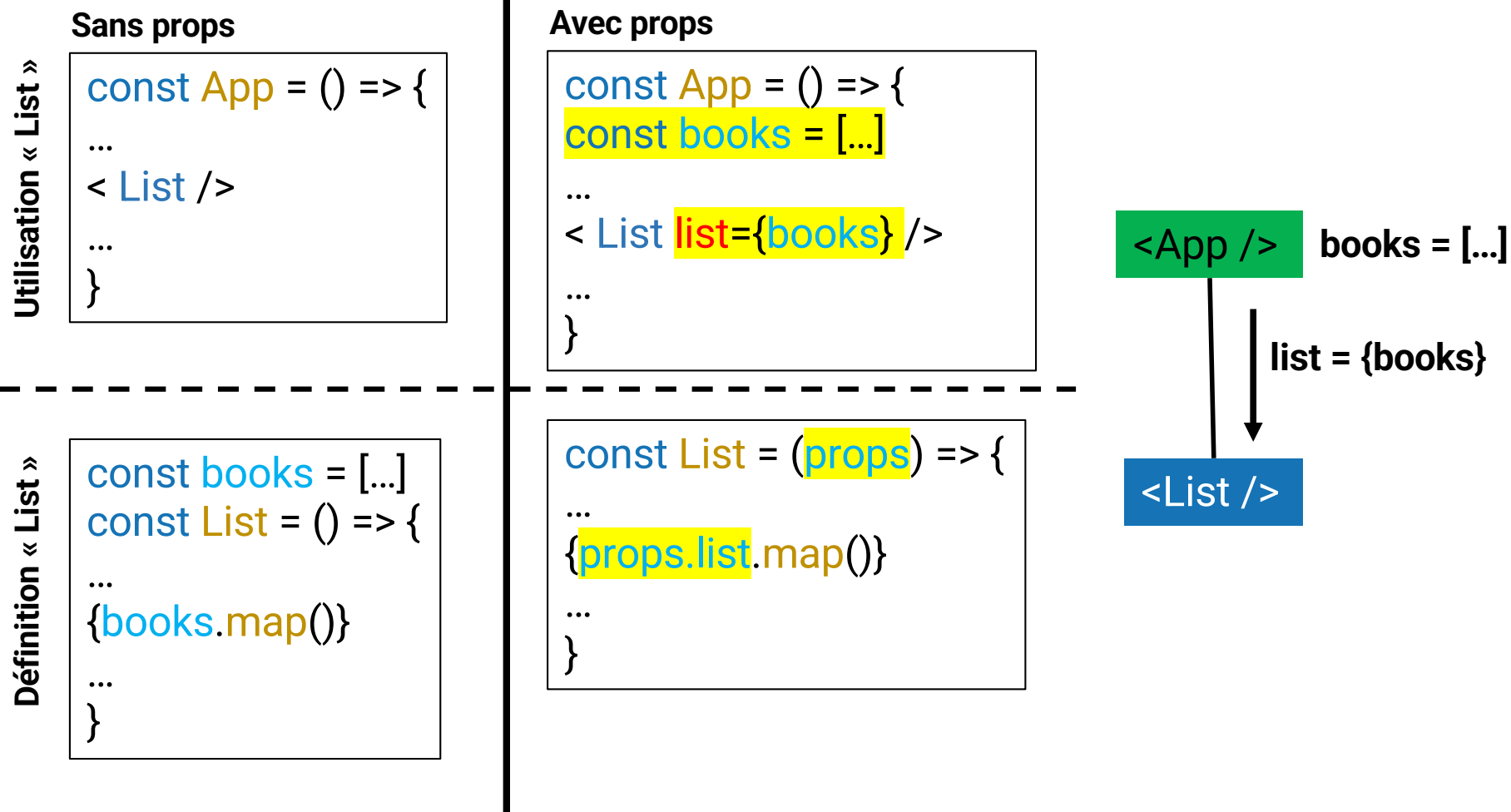
```
function App() {  
  return (  
    <>  
      <p>Hello World!</p>  
      <p>Let's learn React!</p>  
    </>  
  );  
}
```

Valide

Les **props (properties)** sont utilisés **pour transmettre des informations** au sein de la hiérarchie des composants (Parent -> Enfant).



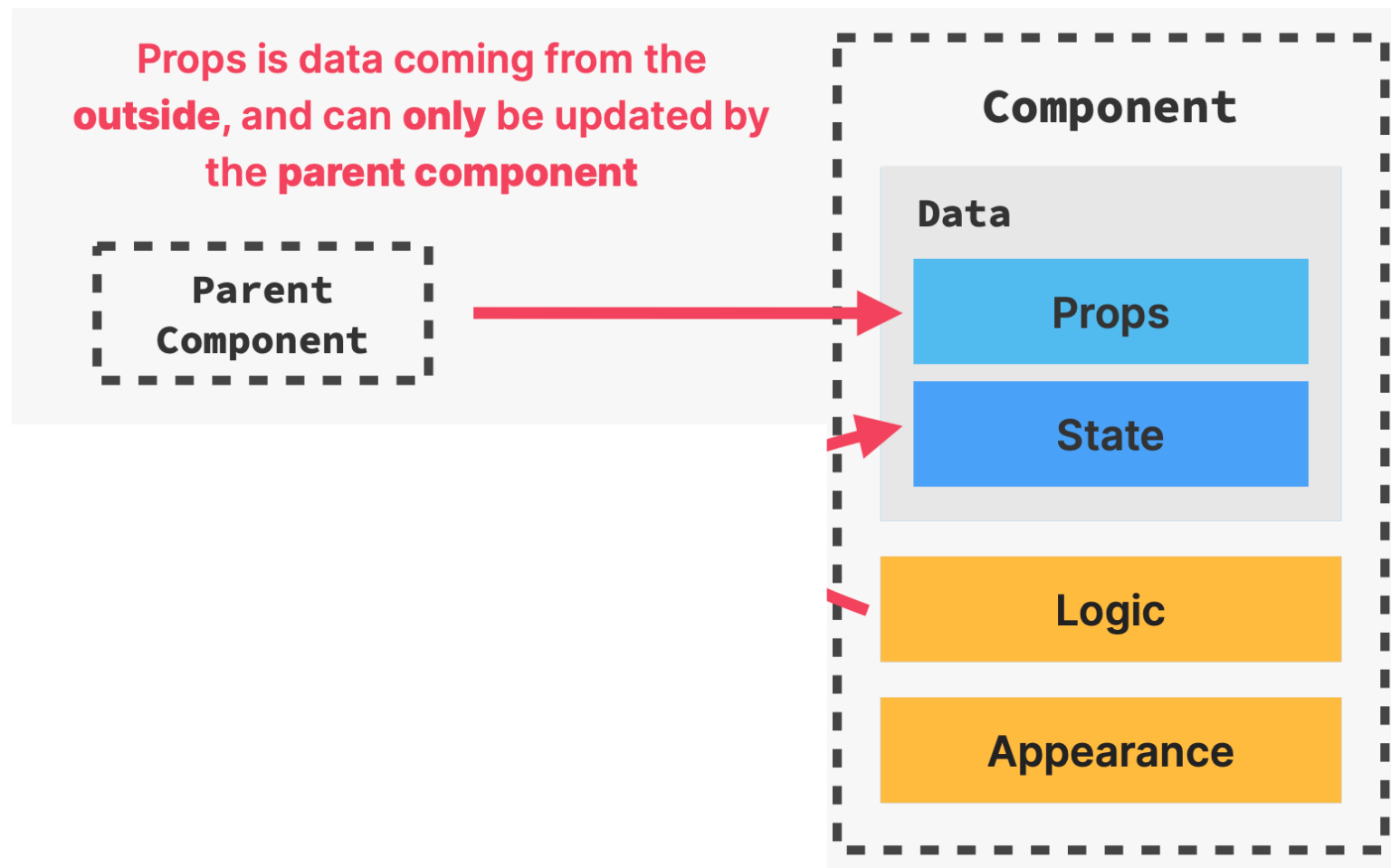
Les informations (props) ne peuvent être transmises que d'un composant parent à un composant enfant et non l'inverse.



```
function App() { Show usages ⓘ HeH-Gianni *  
  const welcome : {greeting: string, title: string} = {  
    greeting: 'Hey',  
    title: 'React'  
  };  
  const books : [{title: string, url: string, ...} = [ 2 elements... ];  
  return (  
    <>  
      <h1> {welcome.greeting} {welcome.title}</h1>  
      <Search/>  
      <hr />  
      <List list={books}/>  
    </>  
  )  
}  
export default App Show usages ⓘ HeH-Gianni
```

```
const List = (props) => { Show usages ⓘ HeH-Gianni *  
  return (  
    <ul>  
      {props.list.map((book) => (  
        <li key={book.objectID}>  
          <span><a href={book.url}>{ book.title}</a></span>  
          <span> {book.author}</span>  
          <span> ({book.num_comments})</span>  
          <span> [{book.points}]</span>  
        </li>  
      )}}  
    </ul>  
  );  
}  
export default List; Show usages ⓘ HeH-Gianni
```

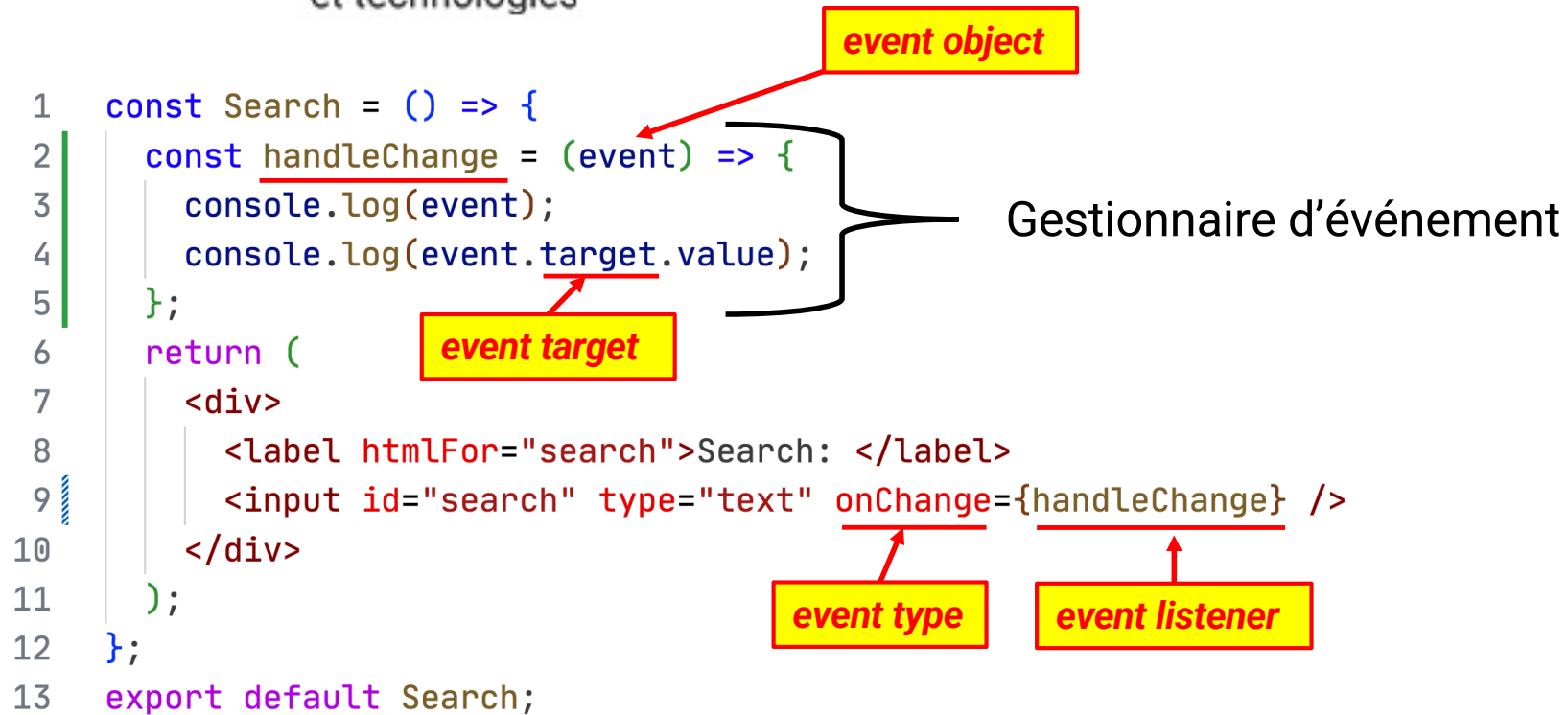
- Le nom du paramètre (props) est libre (par convention, props).
- Cet argument props automatiquement fourni contiendra toujours un objet (React passe un objet comme valeur pour cet argument)
- les propriétés de cet objet seront les "attributs" que vous avez ajoutés à votre composant dans le code JSX où le composant est utilisé.



Partie 4: événements et state

Rappel :

- **Objet événement (*event object*)**
Cet objet est associé à un événement particulier et contient des détails sur cet événement. Les objets d'événement sont transmis en tant qu'argument à la fonction de gestion d'événement.
- **Cible de l'événement (*event target*)**
C'est *l'objet* sur lequel l'événement s'est produit ou avec lequel l'événement est associé (bouton, champ texte, ...)
- **Gestionnaire d'événement ou écouteur d'événement (event handler, or *event listener*)**
Cette *fonction* gère ou répond à un événement.
- **Type d'événement (*event type*)**
C'est une *chaîne de caractères* qui indique le type d'événement (click, load, ...)



Remarque :

- `handleChange()` : **invoker** la fonction => récupérer la valeur de retour
- `handleChange` : retourner la **référence** de la fonction

```
1  const Search = () => {  
2    let searchTerm = '';  
3    const handleChange = (event) => {  
4      searchTerm = event.target.value;  
5    };  
6    return (  
7      <div>  
8        <label htmlFor="search">Search: </label>  
9        <input id="search" type="text" onChange={handleChange} />  
10       <p>Recherche de <strong>{searchTerm}</strong></p>  
11     </div>  
12   );  
13 };  
14 export default Search;  
15
```

Les valeurs saisies n'apparaissent pas, car le composant n'est pas réaffiché (re-render) !!!

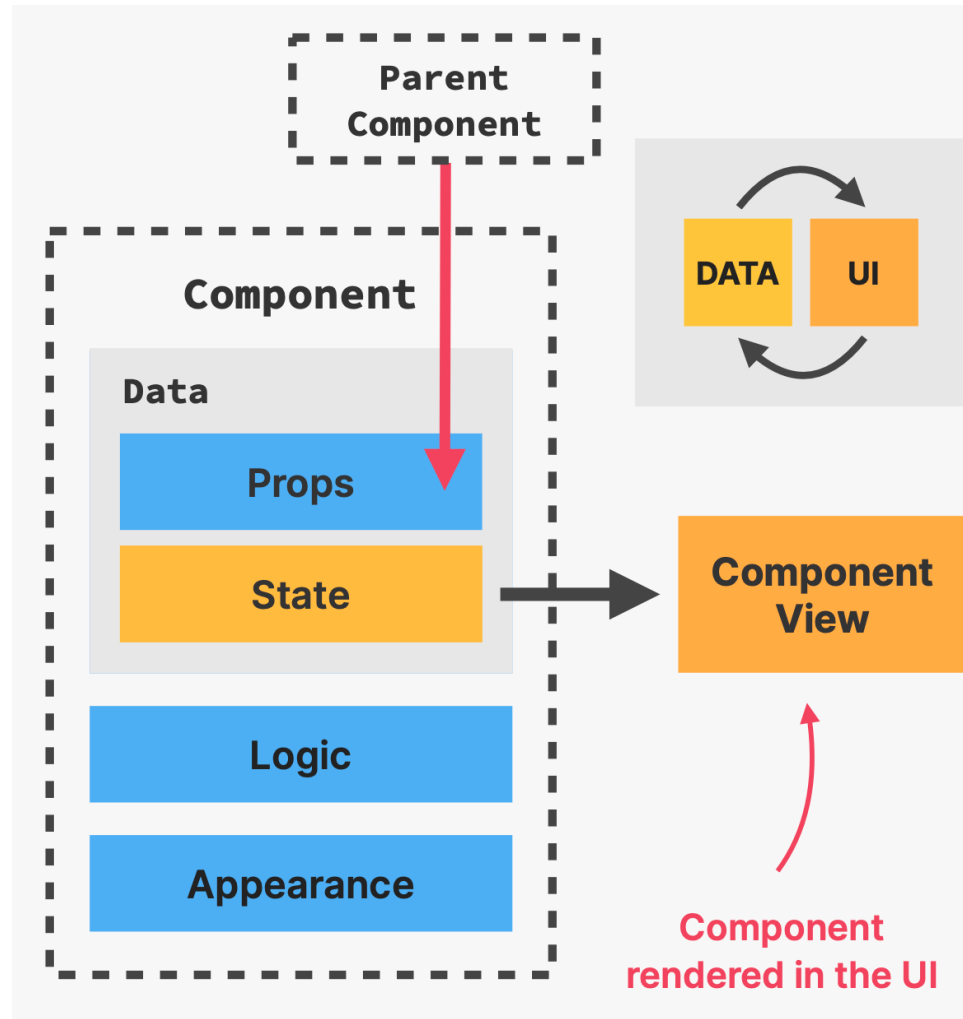
Pour que le composant se redessine lorsque la valeur de la variable « *searchTerm* » change => celle-ci doit être définie comme une **variable d'état**.

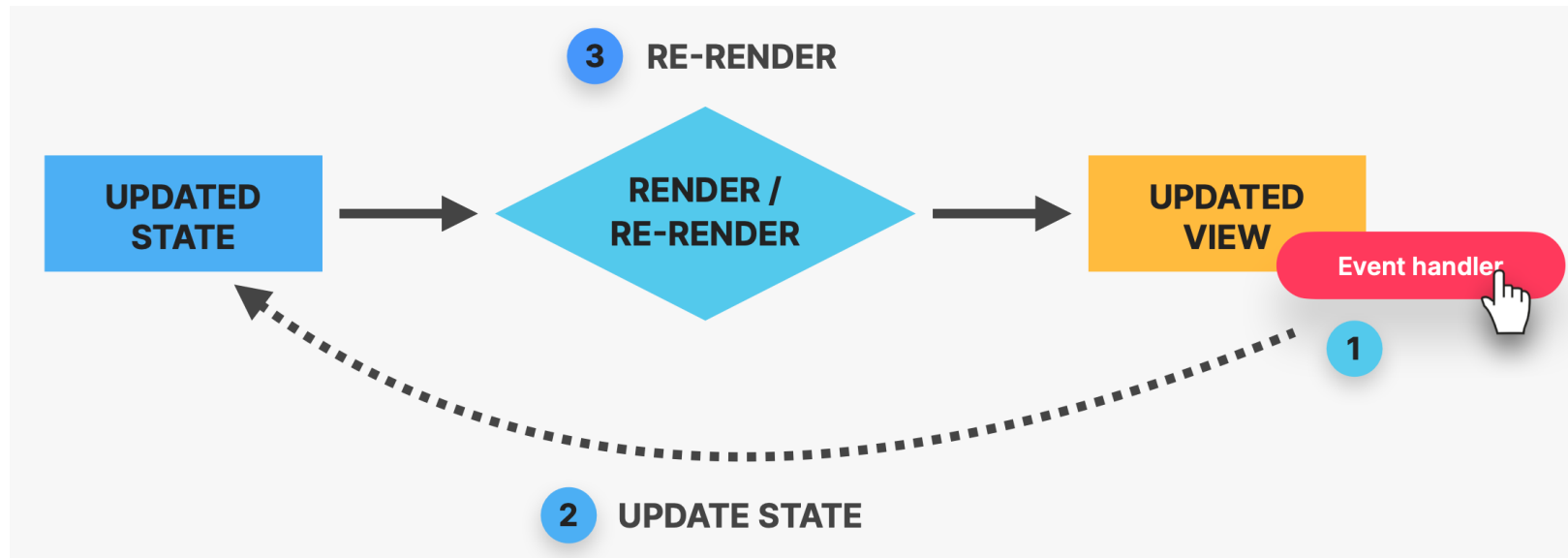
L'état est une donnée qui, lorsqu'elle est modifiée, doit **forcer React à réévaluer** un composant et à mettre à jour l'interface utilisateur si nécessaire.

Hooks :

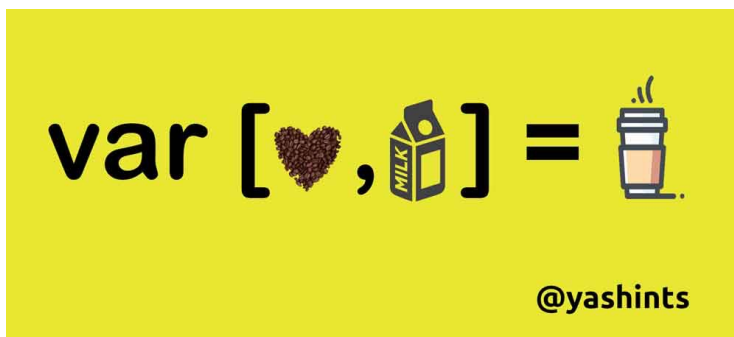
Ce sont des **fonctions spéciales** qui ne peuvent être utilisées qu'à **l'intérieur des composants** React.

Hook *useState()* permet à un composant de définir et de gérer un état lié à ce composant.





Affectation par décomposition (destructuring) - Tableaux



<https://yashints.dev/blog/2018/09/12/destructuring-in-js/>

```
const toto = ["un", "deux", "trois"];

// sans utiliser la décomposition
const un = toto[0];
const deux = toto[1];
const trois = toto[2];

// en utilisant la décomposition
const [un, deux, trois] = toto;

console.log(` ${un} et ${deux} et ${trois} `);
```

```
const [searchTerm, setSearchTerm] = useState("");
```

Affectation de décomposition

Valeur de l'état initiale

`useState("")` -> fonction qui retourne un tableau à 2 éléments.

Le **premier élément** du tableau (`searchTerm`) est toujours la **valeur** de l'état actuel.

Le **second élément** du tableau (`setSearchTerm`) est une **fonction** que vous pouvez appeler pour **mettre à jour l'état**. => ça permet d'informer React qu'un état a changé.

```
1 | import { useState } from 'react'; ← Importer la fonction « useState »
2 | const Search = () => {
3 |   const [searchTerm, setSearchTerm] = useState('');
4 |   const handleChange = (event) => {
5 |     setSearchTerm(event.target.value);
6 |   };
7 |   return (
8 |     <div>
9 |       <label htmlFor="search">Search: </label>
10 |      <input id="search" type="text" onChange={handleChange} />
11 |
12 |      <p>Recherche de <strong>{searchTerm}</strong></p>
13 |    </div>
14 |  );
15 | };
16 | export default Search;
```